

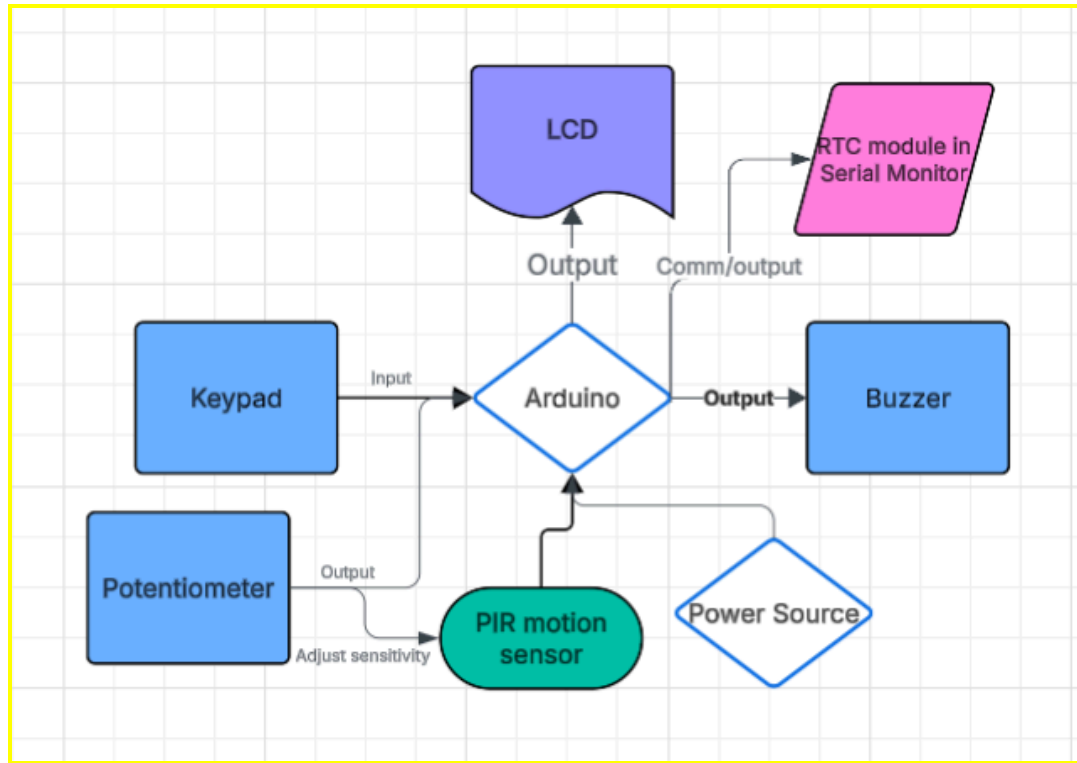
1. Project Motivation/Introduction

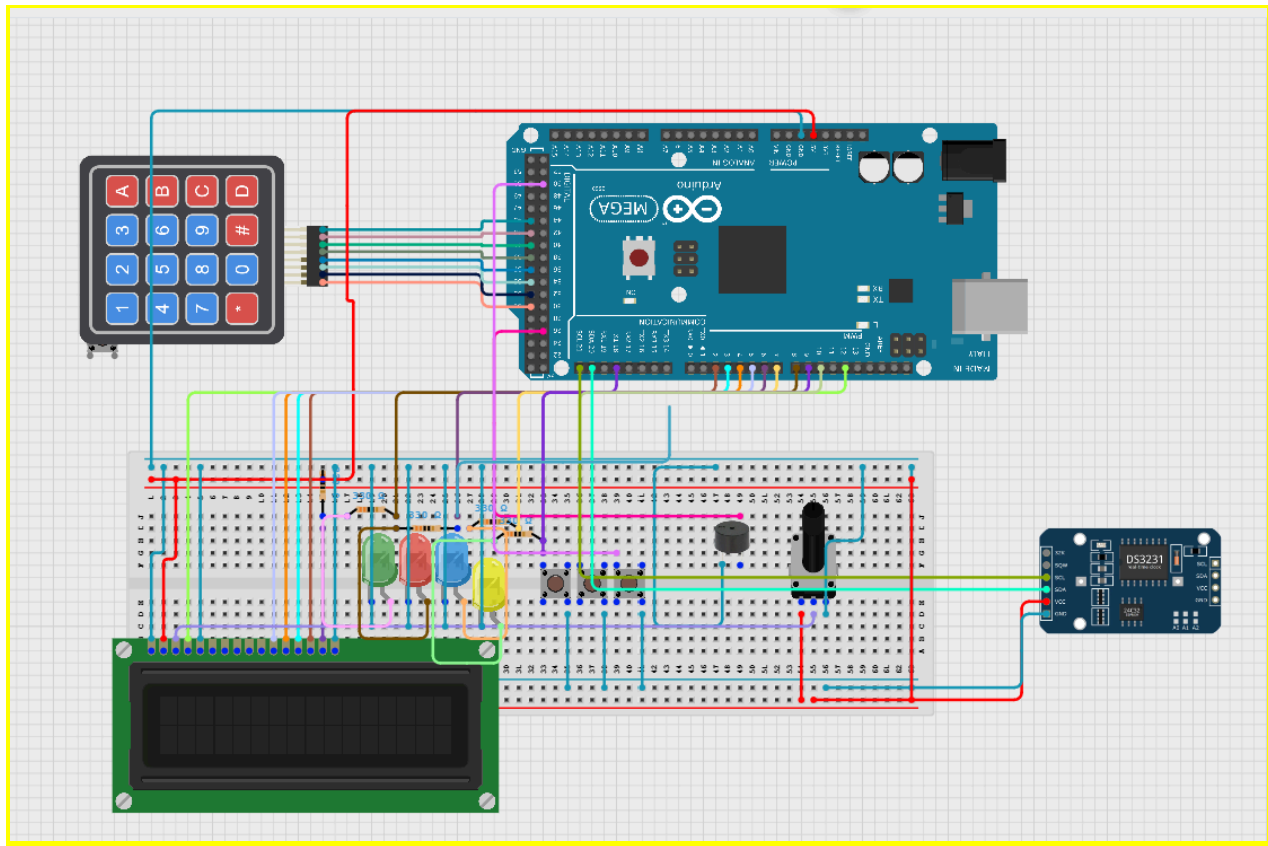
My system addresses the need for a security system for a wide range of tasks, from business centers, to home security. This system is aimed towards any individual or company looking for a high level and advanced physical contact security system with special lockout features. This embedded system is a reasonable solution to an ongoing problem as threats to security and break-ins are becoming more prevalent. This system could be mass produced and installed on every entry-point around a building or home.

2. Proposed System Overview

The main feature of this system is to either grant access or deny access to an individual using a keypad as an input module. The first input this system will receive will be a motion detection, taken in from a motion sensor. This is to not only ensure ease of access for the intended user but heightened security for a potential intruder. The LED screen will start off and then turn on with the on button and show the security company logo "BIDDLE Industries". Following the detection of motion, an LED screen will prompt them to input their password using a physical keypad. The password of the system will be hardcoded to ensure no one unauthorized can change it. A potentiometer can also be used to modify the screen's brightness in order to see it more

clearly at different times of the day or light levels.





3. Expected System Behavior (High-Level)

When this system is turned on, it will sit in an idle state showing “BIDDLE Industries” waiting for the motion sensor to detect anybody which will then prompt the LED screen to ask for a user's password. If a user fails to input the correct password, the LED will prompt them with $\frac{1}{3}$ failed attempts. If the user again inputs the wrong password, the screen will give them 1 more chance before flashing an error message which can only be reset with the reset button. This error will also occur if the ultrasonic sensor malfunctions or something gets too close to it and it reads a negative number, both will be differentiated by different error messages. If the user inputs the correct password, the screen will grant them access with a message. When the system is turned off, all components will shut down and nothing can be accessed. The only changes from the

physical environment that will alter the system would be physical movement into the range of the motion sensor. As all of this is happening the RTC module will show in the serial monitor when each state is changed and why.

3.1. Module Design

Sensor reading module: This module is responsible for collecting all environmental and analog input data. The PIR motion sensor and potentiometer are the sensors used. The motion sensor detects motion and gives a high or low signal. The potentiometer gives analog values to change the sensitivity of the LCD screen.

Input module: Handles all user inputs to the system. The keypad, on button, off button and reset button are all components in this module. The keypad detects input and the other button works directly with the control logic module to trigger state changes.

Control logic module: This is the core decision-making module of the system. It Implements the system's state machine (OFF, IDLE, ACTIVE, ERROR) and Processes input from sensors and user inputs. The module receives data from Sensor and Input modules.

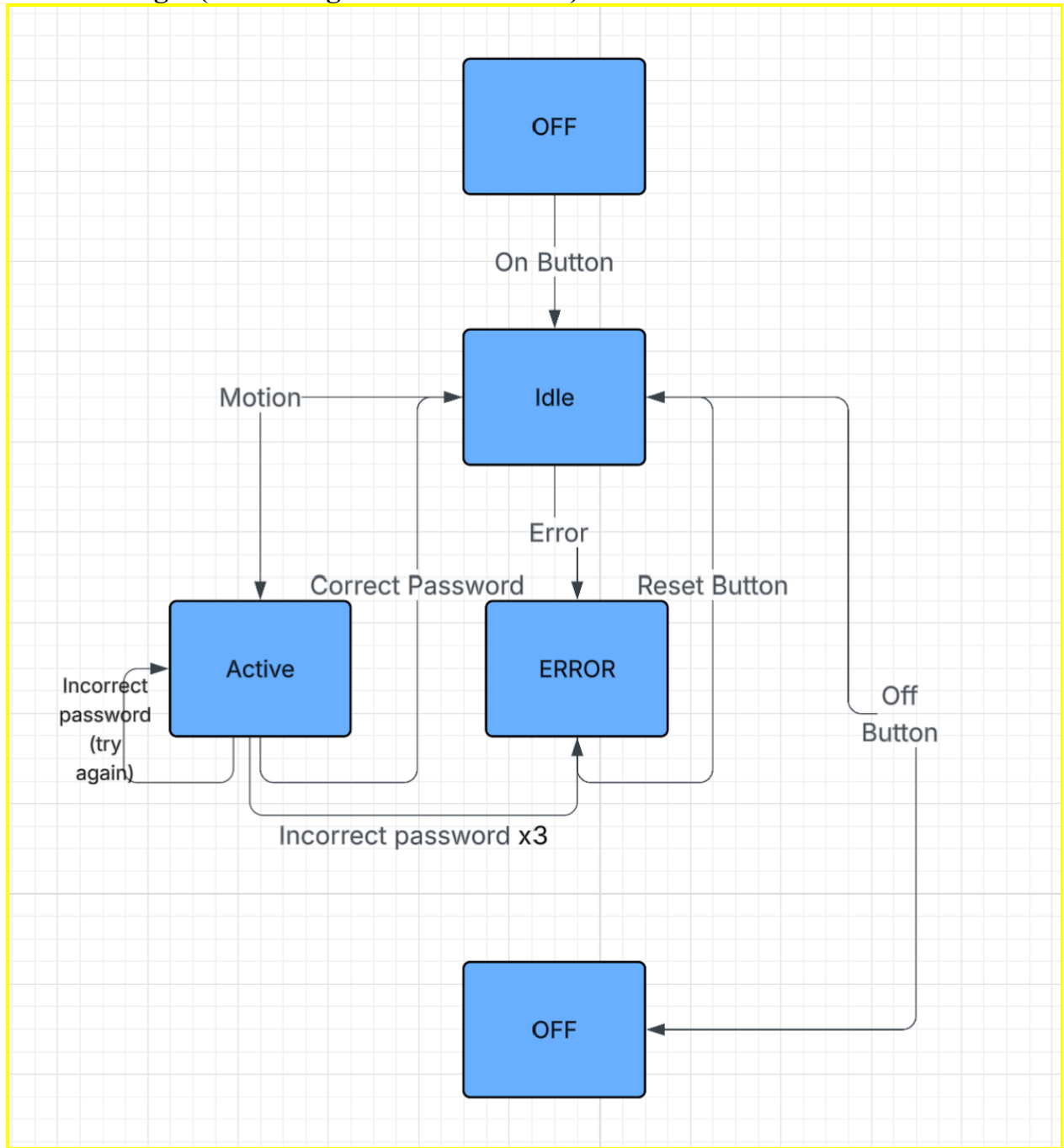
Display module: This module provides real-time feedback to the user through the LCD. It displays the current system state, prompts for password input, and shows messages such as "ACCESS GRANTED" or "ERROR - LOCKOUT" or ERROR - SENSOR FAIL The display updates dynamically based on commands from the control logic module.

Actuator module: The actuator module performs the physical action of the system, in this case it is the active buzzer. Which is activated and stays on when an error occurs and is a feedback sound for inputting a password on the keypad.

Led indicator module: This module visually indicates the current system state using four LEDs. Each LED corresponds to a specific state (OFF, IDLE, ACTIVE, ERROR), with only one LED active at a time. The LEDs are updated whenever the system transitions between states.

Communication Module: This module logs important system events to the Serial Monitor using time data from the RTC module. It records state transitions, user actions, actuator events, and errors with timestamps. This allows for real-time monitoring and debugging of system behavior.

3.2 Control Logic (State Diagram or Flowchart)



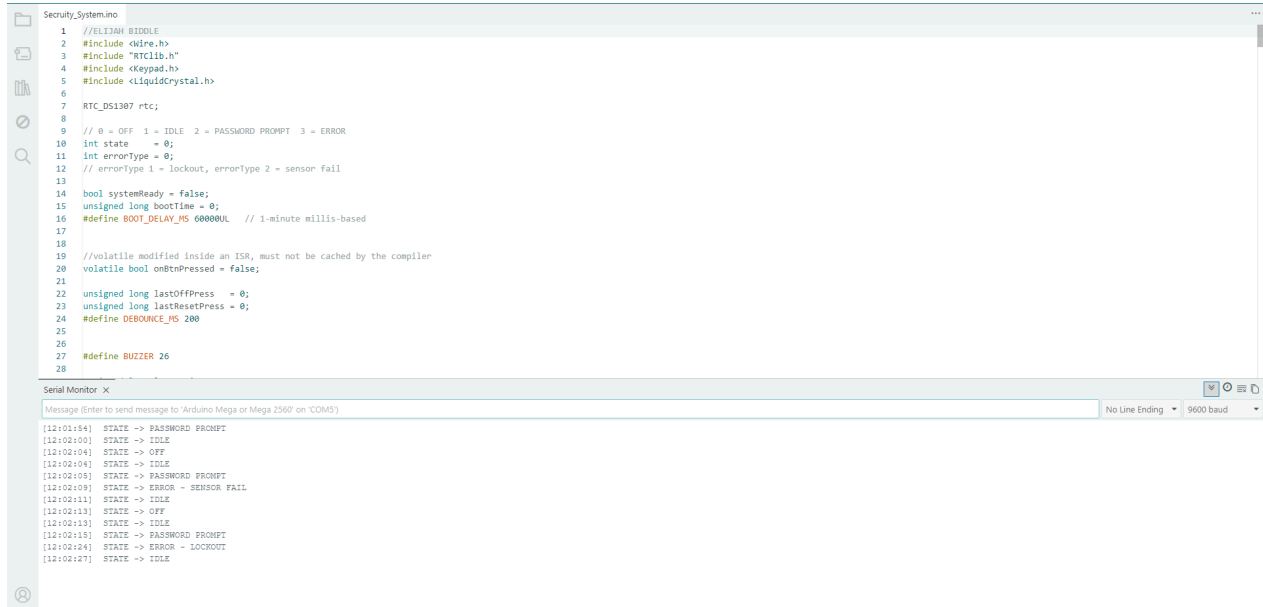
4. Environmental & Energy Impact

- **Energy Efficiency:** The system will stay in an idle state, which will not consume as much power as an active state, until the motion sensor is activated, or the off button is pressed in which case the system will turn off.
- **Design Safety:** All wires and exposed elements will be safely secured and hidden so they pose no danger to the user. If an error were to occur, the system would instantly shut off to avoid harm.
- **Affordability:** System is made from reusable and common electrical components so they are easy and cost effective to fix.
- **Sustainability:** Elements of the system are high quality plastic and metal components.
- **Accessibility:** LED screen and motion sensor allows for ease of access while having a physical input module (keypad) for instant feedback. The motion sensor can be adjusted based on users preference for more modularity.

4. Results

Video Shared

Serial Monitor:



The screenshot displays the Arduino IDE interface. The main window shows the code for `Security_System.ino`. The code includes headers for `wire.h`, `RTClib.h`, `Keypad.h`, and `LiquidCrystal.h`. It defines an `RTC_DS1387` object and sets up variables for system state, error type, boot time, and debounce delay. The code also includes a volatile boolean for button press and unsigned long variables for last off and last reset presses. A buzzer is defined with a pin number of 26. The Serial Monitor at the bottom shows the output of the program, displaying the state of the system at various time intervals.

```
1 //ELIJAH BIDDLE
2 #include <Wire.h>
3 #include "RTClib.h"
4 #include <Keypad.h>
5 #include <LiquidCrystal.h>
6
7 RTC_DS1387 rtc;
8
9 // 0 = OFF 1 = IDLE 2 = PASSWORD PROMPT 3 = ERROR
10 int state = 0;
11 int errorType = 0;
12 // errorType 1 = lockout, errorType 2 = sensor fail
13
14 bool systemReady = false;
15 unsigned long bootTime = 0;
16 #define BOOT_DELAY_MS 60000UL // 1-minute millis-based
17
18
19 //volatile modified inside an ISR, must not be cached by the compiler
20 volatile bool onBtnPressed = false;
21
22 unsigned long lastOffPress = 0;
23 unsigned long lastResetPress = 0;
24 #define DEBOUNCE_MS 200
25
26
27 #define BUZZER 26
28
```

Serial Monitor X

Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM5')

No Line Ending 9600 baud

```
[12:01:54] STATE -> PASSWORD PROMPT
[12:02:00] STATE -> IDLE
[12:02:04] STATE -> OFF
[12:02:04] STATE -> IDLE
[12:02:05] STATE -> PASSWORD PROMPT
[12:02:09] STATE -> ERROR - SENSOR FAIL
[12:02:11] STATE -> IDLE
[12:02:13] STATE -> OFF
[12:02:13] STATE -> IDLE
[12:02:18] STATE -> PASSWORD PROMPT
[12:02:24] STATE -> ERROR - LOCKOUT
[12:02:27] STATE -> IDLE
```

5. Conclusion

I was able to get my embedded system to work which in and of itself is a huge accomplishment that I am proud of. Overall there were 14 input and output devices used along with countless wires and resistors to make this system work. This is by far the biggest project I have ever worked on, which actually posed a limitation itself. The half breadboard became a little cramped and the wire sizing between each component was not consistent or entirely accurate. This was not an issue I had foreseen, but in the future I would like to have more spaced out components and take more time to choose how I wire things accordingly.